INTERVIEW PREPARATION GUIDE

Overview

An interview at this company is relatively straightforward and typically lasts up to one hour. However, being prepared is critical for success, the details of which can be found in the slides below.

Usually, the interview comprises three sections: a self-introduction, technical questions, and live coding. It's essential to be prepared for each of these segments. The slides below consolidate the information you need to effectively prepare and excel in the interview.

Interview Preparation

Before proceeding with the interview, please adhere to the following readiness checklist:

- Make sure that you can share your screen.
- Your video camera and microphone should function properly throughout the session (verify both beforehand).
- You should have an IDE installed on your device, and must be capable of compiling code during the interview.
- Use of mobile devices is strictly prohibited during the interview.
- Use of second screens is not permitted.
- Please tidy up your screen and browser; close all tabs, particularly AI helpers such as ChatGPT, prior to the interview.
- Searching for answers on Google or utilizing tools such as ChatGPT during the interview is strictly prohibited. Interviewers are highly skilled and can detect any form of dishonesty. However, if you need to look up specific coding information during the live-coding session, you may request permission from the interviewers to do so.
- Feel free to join interview link earlier and check everything.
- **Never join the interview from a mobile device!**
- Never be late; if you might be unable to attend, notify your coordinator well in advance.

Internal

Best Practices

- Don’t hesitate to ask for a question to be repeated if you aren’t sure about it.
- Write everything directly on the screen, not on paper. For instance, if you’re asked to solve a coding challenge, you can write the task description on the screen. This way, the interviewers can verify your understanding.
- Honesty is key. If you don’t know the answer to a question, don’t attempt to fabricate a response. It’s perfectly okay to admit that you lack experience in a specific area.
- Don’t aim to craft the perfect solution during the live coding session. A compilable solution is 95% of the success. If performance or stylistic adjustments are necessary, the interviewers will let you know. Initially, focus on resolving the task in any possible manner.

Internal

Self-Introduction

Keep it concise and relevant, preferably within three minutes. It's beneficial to emphasize relevant expertise from your past projects, such as:

- Networking experience
- Mastery of protocols
- Knowledge of network topology
- Familiarity with Docker/K8s/K3s
- Troubleshooting skills
- Cloud experience (any platform)
- Certificates

Technical Questions

The interviewers may ask various questions related to the position you're applying for. Make sure to thoroughly read the job description prior to the interview to better understand potential queries. Here are a few examples of typical questions for C/Python/Golang positions:

- What are the pros and cons of microservices architecture?
- Can you explain various data structures?
- What unit testing tools have you used and what is your experience with them?
- Can you discuss design patterns and principles?
- What are "Routines" in Golang?
- Can you describe your experience troubleshooting networks?
- What is a "Binary tree"?

Internal

Networking Knowledge

This is a networking company, and for most positions, it is required to have some fundamental knowledge in networking. Please review the **networking.md** document for detailed information on all required topics. You should have a basic understanding of the following areas:

1. TCP/IP Fundamentals and associated protocols (HTTP, HTTPS, FTP, etc)
2. UDP Protocol: Concepts and differences from TCP/IP
3. DNS: Functionality, Records, and Queries
4. DHCP: Understanding of DHCP operations (DORA process)
5. Major Network Protocols such as ARP, ICMP, OSPF and BGP
6. Network Topologies: Star, Bus, Ring, Mesh and Hybrid
7. Network Infrastructure and Hardware: Switches, Routers, Hubs, Modems and Network Interfaces
8. VPN, SSL, TLS, and other encryption technologies
9. Network Security principles (Firewalls, IDS/IPS)
10. Subnetting and IP Addressing (IPv4 and IPv6)
11. Troubleshooting tools: Ping, Traceroute, Netstat, nslookup, Wireshark, etc.
12. Network Monitoring Tools
13. Load Balancing and Fail-over concepts
14. Cloud Networking concepts (VPC, Subnets, etc.)
15. Network Automation and Orchestration (Ansible, Puppet, etc.)
16. QoS (Quality of Service) concepts
17. Software Defined Networking (SDN)
18. Web Services/APIs and REST Protocol
19. Socket programming in Python
20. HTTP requests in Python (requests, urllib libraries)
21. Python network libraries (paramiko, scapy, etc.)

Internal

Coding Challenge

The slides below contain examples of coding challenges from previous interviews for various skill sets. Ensure to attempt all challenges relevant to your expertise. If most challenges are too difficult, or if you’re unable to solve a challenge within 30 minutes, it may be advisable to decline the interview.

GO INTERVIEW CODING CHALLENGES

Internal

Challenge 1 · Username Generator

- **Goal**: Produce an 8-character alphanumeric username made of two 4-character segments (first-name + last-name). Implement a concise Go program that generates such usernames.
- **Notes**: Decide how to handle non-alphanumeric input and whether to seed randomness for reproducible tests.

Challenge 2 · Concurrent Job Dispatcher

- **Scenario**: Build a dispatcher that assigns a list of jobs to workers concurrently.
- **Requirements**:
  - Spawn `N` worker goroutines.
  - Distribute `M` jobs through a channel.
  - Each worker prints when it starts and finishes a job.
  - Simulate processing time with `time.Sleep`.
  - Use `sync.WaitGroup` to wait for all jobs to finish.
  - Ensure no job runs more than once.
- **Example Input**:

```
jobs := []Job{
    {ID: 1, Duration: 2 * time.Second},
    {ID: 2, Duration: 1 * time.Second},
    {ID: 3, Duration: 3 * time.Second},
    // ...
}
```

- **Typical Output (order may vary)**:

```
Worker 1 started job 2
Worker 3 started job 1
Worker 2 started job 3
Worker 1 finished job 2
Worker 3 finished job 1
Worker 2 finished job 3
All jobs completed.
```

Challenge 3 · Minimum Shipping Capacity

- **Prompt**: For `n` warehouses containing `piles[i]` packages, and a single truck with `h` hours before a deadline, find the minimum capacity `k` (packages/hour) that ships everything in time. The truck visits one warehouse per hour and, if a warehouse has fewer than `k` packages, ships all of them then idles for the rest of the hour.
- **Output**: Return the smallest integer `k` that satisfies the requirement.
- **Examples**:
  - `piles = [3, 6, 7, 11], h = 8 → k = 4`
  - `piles = [30, 11, 23, 4, 20], h = 5 → k = 30`
- *Note*: A second version of the prompt describes “items” instead of “packages”; the algorithm is identical.

Internal

Challenge 4 · Issue Tracking Service

- **Goal**: Design a lightweight service similar to Jira.
- **Tasks**:
  - Define an `Issue` struct with `ID`, `Title`, `Status`, `Assignee`, and `CreatedAt`.
  - Implement `CreateIssue(title string, assignee string) *Issue` and `ListIssuesByAssignee(assignee string) []*Issue`.
  - Store issues in `map[int]*Issue` with incremental IDs.
  - Ensure concurrency safety; multiple goroutines may invoke the functions.
  - Demonstrate usage in `main()`.

Internal

Challenge 5 · Custom Map API

- **Prompt**: Implement a map-like data structure in Go that provides `get()` and `put()` semantics. Decide how to address collisions, concurrency, and zero-value handling.

Challenge 6 · Rotated Sorted Array Search

- **Goal**: Given a rotated sorted array (e.g. `[4,5,6,7,1,2,3]`) and a target such as `6`, return the index of the target in `O(log n)` time.

Challenge 7 · Ordered Odd–Even Printer

- **Scenario**: Launch two goroutines—one printing odd numbers, the other printing even numbers—so that the combined output is `1,2,3,...,n` in order. Coordinate with channels or another synchronization primitive to prevent out-of-order prints.

Challenge 8 · Sum of Squares via Goroutines

- **Goal**: Start two goroutines: `squares` to compute squares of numbers from `1` to `n`, and `adder` to consume those values and compute their sum. The main function should launch the process, print the total, and exit cleanly once the work is done.

Challenge 9 · Parallel Slice Sum

- **Exercise**: Compute the sum of a slice of integers using multiple goroutines. Use channels (or another coordination mechanism) to gather partial sums into a final result.

Challenge 10 · Random Number Generator Count

- **Prompt**: Use two goroutines that generate random numbers and print how many numbers each routine has produced so far. Include the generated number in the output message.

Challenge 11 · Timed Ping-Pong Game

- **Scenario**: Simulate two players rallying a ping-pong ball for two minutes.
- **Requirements**:
  - Represent each player with a goroutine; use channels to pass the ball.
  - Each player waits a random 0–10 seconds before returning the ball.
  - After two minutes, the rally stops and the player currently holding the ball wins.
  - Manage cancellation with `context.Context`.

Challenge 12 · Longest Substring Without Repeating Characters

- **Problem**: Given a string `s`, determine the length of the longest substring without repeated characters.
- **Example**: `s = "abcabcbb" → length = 3` (substring `"abc"`).

Challenge 13 · Concurrent Word Frequency Counter

- **Objective**: Read multiple text files concurrently, count word frequencies across all files, and print the results sorted by descendingfrequency.
- **Requirements**:
  - Use goroutines and channels for file reading and word counting.
  - Update shared counts safely.
  - Handle errors gracefully.
  - Sort and display the final frequency list.
- **Sample Input (`sample.txt`)**:

```
Go is an open-source programming language created at Google.
It makes it easy to build simple, reliable, and efficient software.
Go is known for its simplicity and efficiency.
The Go programming language is also referred to as Golang.
Go has a strong emphasis on concurrency and performance.
```

Internal

Challenge 14 · Method Receiver Gotcha (Person Age)

- **Prompt**: Analyze what the following Go program does, whether it compiles, and what it prints.

```go
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func main() {
    p := NewPerson("X", 20)
    p.SetAge(25)
    fmt.Println(p.Age)
}

func NewPerson(name string, age int) *Person {
    return &Person{Name: name, Age: age}
}

func (p Person) SetAge(age int) {
    p.Age = age
}
```

- **Explanation**: `NewPerson` returns a pointer to a `Person`. The `SetAge` method uses a value receiver, so calling it on a pointer copies the struct and mutates the copy only. The original `Person` remains unchanged.
- **Compilation**: Yes, it compiles; Go automatically dereferences the pointer when invoking a value-receiver method.
- **Output**: `fmt.Println(p.Age)` prints `20`, not `25`.
- **Takeaway**: Use a pointer receiver (`func (p *Person) SetAge(age int)`) when a method needs to modify the struct's fields.

Internal

Challenge 15 · Binary Search Tree Verification

- **Goal**: Write a function to determine if a binary tree is a valid binary search tree.
- **Requirements**:
  - For each node, all values in its left subtree must be less than the node's value
  - All values in its right subtree must be greater than the node's value
  - This property must hold for every node in the tree
- **Approach**: Use recursion with min/max bounds, or perform in-order traversal to verify ascending order.

Internal

Challenge 16 · Power of Two Check

- **Goal**: Given an integer, determine if it's a power of 2 without using the `pow` function from the math package.
- **Constraints**: Preferably solve without loops.
- **Hint**: A number is a power of 2 if it has exactly one bit set. Use bitwise operations: `n > 0 && (n & (n-1)) == 0`.

Challenge 17 · Highest Sum Consecutive Sequence

- **Goal**: Given a set of non-negative integers, find the highest sum of a consecutive sequence where numbers differ by 1.
- **Requirements**:
  - A sequence is at least 2 numbers that differ by 1 (e.g., `[5, 6, 7]` or `[9, 10]`)
  - Numbers in the sequence don't need to be adjacent in the input array
- **Examples**:
  - Input: `[3, 2, 7, 8, 11, 6]` → Output: `21` (sequence `[6, 7, 8]`)
  - Input: `[5, 6, 15]` → Output: `11` (sequence `[5, 6]`)
- **Approach**: Sort the array first, then find consecutive runs and calculate their sums.

Challenge 18 · Array Rotation by Index

- **Goal**: Given an array and an index, rotate the array by that index position.
- **Example**: Input `[5, 7, 8, 4, 9, 1]`, index `3` → Output `[9, 1, 5, 7, 8, 4]`
- **Constraint**: Ideally implement in-place without creating an additional array.
- **Hint**: Use array reversal technique: reverse the entire array, then reverse the first k elements, then reverse the remaining elements.

Internal

Challenge 19 · Max-Min Alternating Sort

- **Goal**: Given an ordered array, re-sort it so elements alternate between maximum and minimum values.
- **Pattern**: First maximum, first minimum, second maximum, second minimum, etc.
- **Example**: Input `[2, 3, 5, 7, 8, 9]` → Output `[9, 2, 8, 3, 7, 5]`
- **Constraint**: Ideally implement in-place without creating an additional array.
- **Approach**: Use two pointers (start and end) and alternate picking from each end.

Internal

Challenge 20 · Expression Calculator

- **Goal**: Given a string expression containing numbers and arithmetic operations (only + and −), calculate its value.
- **Constraints**: Do not use ready-made parsers.
- **Examples**:
  - `"2+6+4"` → `12`
  - `"20-10-6"` → `4`
  - `"123+200"` → `323`
- **Approach**: Parse character by character, accumulate numbers, and apply operations as they're encountered.

Challenge 21 · Goroutine Number Doubler

- **Goal**: Pass a number to a goroutine, have it double the value, pass it back to main, and print the result.
- **Requirements**:
  - Explain your implementation approach
  - Discuss possible ways to solve the task (buffered vs unbuffered channels, sync primitives, etc.)
- **Considerations**: Demonstrate understanding of goroutine communication patterns and synchronization.

Internal

Challenge 22 · Producer-Consumer with Timed Producer

- **Goal**: Implement a producer-consumer pattern where:
  - **Producer**: Generates incremental numbers every 700 milliseconds
  - **Producer**: Exits on its own after 4 seconds
  - **Consumer**: Prints each generated number
  - **Consumer**: Exits after printing all numbers
- **Requirements**:
  - Use channels for communication
  - Use `time.Ticker` or `time.Sleep` for timing
  - Ensure clean shutdown with `sync.WaitGroup` or channel closure
  - Producer should generate approximately 5-6 numbers (4000ms / 700ms)
- **Considerations**: How to signal completion, when to close channels, graceful shutdown.

Internal

Challenge 23 · String to Letter Codes Decoder

- **Goal**: Given a string of digits, find all possible alphabet codes it can generate. Letter mapping: a=1, b=2, c=3…z=26.
- **Requirements**:
  - Generate all valid decodings
  - Return count of valid decodings
  - Print all possible decoded strings
- **Example**:
  - Input: `"1123"`
  - Output:
    - `"aabc"` (a=1, a=1, b=2, c=3)
    - `"kbc"` (k=11, b=2, c=3)
    - `"alc"` (a=1, l=12, c=3)
    - `"aaw"` (a=1, a=1, w=23)
    - `"kw"` (k=11, w=23)
  - Count: 5
- **Approach**: Use recursion or dynamic programming to explore all valid digit groupings.

Internal

Challenge 24 · Arrange Zeros and Ones

- **Goal**: Given an integer array of size n filled with zeros (0) and ones (1), arrange all zeros on the left and all ones on the right.
- **Example**: Input `[0,1,0,1,0,1,0,1]` → Output `[0,0,0,0,1,1,1,1]`
- **Constraint**: Solve in-place with O(n) time complexity and O(1) space complexity.
- **Approach**: Use two-pointer technique or partitioning similar to Dutch National Flag algorithm.

Internal

Challenge 25 · Minimum Days to Assemble Products

- **Goal**: Given an array `readyDay` where `readyDay[i]` represents the day on which the i-th component becomes available on an assembly line, determine the minimum number of days needed to assemble `m` finished products.
- **Requirements**:
  - Each product requires `k` consecutive components from the line
  - Each component can be used only once
  - A component can only be used on or after its ready day
  - Return `-1` if it's not possible to assemble all `m` products
- **Example**:
  - Input: `readyDay = [1, 10, 3, 10, 2], m = 3, k = 1`
  - Output: `3`
  - Explanation: With `k=1`, each product needs 1 component. By day 3, components at indices 0, 2, and 4 are ready (days 1, 3, 2), allowing us to assemble 3 products.
- **Approach**: Use binary search on the answer (number of days). For each candidate day `d`, check if you can form `m` groups of `k` consecutive components where all components in each group have `readyDay[i] <= d`. Use a greedy approach to count valid groups.
- **Complexity**: O(n log(max(readyDay))) time where n is the length of the array.